HARAMAYA UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE

CoSc3053: 90-Day Java Mastery Challenge

DAY 6 CHALLENGE WORKSHEET: CLASSES, OBJECTS & ENCAPSULATION INVARIANTS

Student Name:		**Student ID:**	
Section / Group:		**Submission Date:**	
IDE & JDK Version:	IntelliJ / JDK 25	**Score / Grade:**	**/ 10.0 Marks**

1. Challenge Overview & Architectural Objectives

Day 6 focuses on the core foundations of Object-Oriented Programming (OOP) in Java: Class Blueprints, Object Instantiation, Encapsulation Invariants, Defensive State Validation, and Atomic Multi-Object Transactions. In this challenge worksheet, you will complete incomplete starter code templates (containing explicit // TODO markers) to build a fully robust BankAccount system, an AccountTransferService, and a BankPortfolio analytics manager.

OOP Concept	Architectural Invariant	Implementation Strategy
Encapsulation	State shielding from direct external corruption	Private instance fields + Controlled Public Accessors/Mutators
Constructor Validation	Guaranteed object validity at creation time	Throw IllegalArgumentException on null, blank, or invalid inputs
Defensive Mutators	Business logic rules enforced on state changes	Validate deposit > 0 and withdraw amount <= balance before mutation
Atomic Transfer	All-or-nothing multi-object transaction safety	Execute withdrawal first; proceed to deposit only if withdrawal succeeds

2. Coding Challenges & Incomplete Starter Code

Complete the following three coding challenges. You are provided with incomplete starter code containing explicit // TODO markers. Read the requirements carefully and write robust, production-ready Java code to pass all execution checks.

Challenge 1 (Core - Easy): Encapsulated BankAccount Class with Validation

Requirements:

1. Declare private fields: accountNumber (String, read-only), accountHolder (String), and balance (double).
2. Implement a 3-arg constructor validating that accountNumber and accountHolder are non-null/non-blank, and initialBalance >= 0.0.
3. Implement deposit(double amount): Accepts only amounts > 0.0. Increases balance and returns true.
4. Implement withdraw(double amount): Accepts only amounts > 0.0 AND amount <= balance. Decreases balance and returns true.
5. Provide getter methods for all fields, but NO public setter for balance (mutations allowed ONLY via deposit/withdraw).

```
package edu.haramaya.cosc3053.day6;

public class BankAccount {
    // // TODO 1.1: Declare private encapsulated instance fields
    // private final String accountNumber;
    // private String accountHolder;
    // private double balance;

    // // TODO 1.2: Implement 3-arg constructor with defensive validation
    public BankAccount(String accountNumber, String accountHolder, double initialBalance) {
        // Check if accountNumber is null or blank -> throw IllegalArgumentException
        // Check if accountHolder is null or blank -> throw IllegalArgumentException
        // Check if initialBalance < 0.0 -> throw IllegalArgumentException
        // Assign instance fields...
    }

    // // TODO 1.3: Implement deposit method with validation
    public boolean deposit(double amount) {
        // Rule: amount must be > 0.0
        // If valid: add to balance and return true
        // Otherwise: print error message and return false
        return false; // Replace with your implementation
    }

    // // TODO 1.4: Implement withdraw method with validation & overdraft protection
    public boolean withdraw(double amount) {
        // Rule 1: amount must be > 0.0
        // Rule 2: amount must be <= balance (no negative balance allowed!)
        // If valid: subtract from balance and return true
        // Otherwise: print descriptive error message and return false
        return false; // Replace with your implementation
    }
```

```
    // // TODO 1.5: Implement Getters & Controlled Summary
    public String getAccountNumber() { return null; }
    public String getAccountHolder() { return null; }
    public double getBalance() { return 0.0; }
}
```

Challenge 2 (Intermediate): Atomic AccountTransferService Engine

Requirements:

1. Implement AccountTransferService.transfer(BankAccount fromAcc, BankAccount toAcc, double amount).
2. Guard against null references (throw IllegalArgumentException if fromAcc or toAcc is null).
3. Guard against self-transfer (fromAcc cannot be the same object or have the same accountNumber as toAcc).
4. Enforce Atomic Execution: Call fromAcc.withdraw(amount) first. ONLY if withdrawal succeeds, call toAcc.deposit(amount).
5. Return boolean indicating overall transfer success or failure.

```
package edu.haramaya.cosc3053.day6;

public class AccountTransferService {
    // // TODO 2.1: Implement atomic transfer method
    public static boolean transfer(BankAccount fromAcc, BankAccount toAcc, double amount) {
        // Check if fromAcc or toAcc is null -> print error & return false
        // Check if fromAcc equals toAcc (same account reference/number) -> print error & return
false
        // Check if amount <= 0.0 -> print error & return false

        // Attempt withdrawal from source account
        // If withdrawal succeeds -> deposit into target account & return true
        // If withdrawal fails -> print transaction aborted & return false

        return false; // Replace with your implementation
    }
}
```

Challenge 3 (Advanced): BankPortfolio Portfolio Analytics & Interest Batch Engine

Requirements:

1. Implement BankPortfolio managing an array BankAccount[] accounts.
2. Implement getTotalLiquidity(): Computes the sum of balances across all non-null accounts.
3. Implement getHighestBalanceAccount(): Iterates through accounts to find and return the BankAccount with max balance.
4. Implement applyInterestToEligible(double minBalanceThreshold, double interestRatePercent): Deposits interest (balance * interestRatePercent / 100) into every account whose balance >= minBalanceThreshold.

```
package edu.haramaya.cosc3053.day6;

public class BankPortfolio {
    private BankAccount[] accounts;
    private int count;

    public BankPortfolio(int capacity) {
        this.accounts = new BankAccount[capacity];
        this.count = 0;
    }

    // // TODO 3.1: Add account to portfolio
    public boolean addAccount(BankAccount account) {
        // Check if account is not null and count < capacity -> add & return true
        return false;
    }

    // // TODO 3.2: Calculate total bank liquidity
    public double getTotalLiquidity() {
        // Sum balances of all non-null accounts
        return 0.0;
    }

    // // TODO 3.3: Find highest balance account
    public BankAccount getHighestBalanceAccount() {
        // Find and return the account with maximum balance
        return null;
    }

    // // TODO 3.4: Apply batch interest to eligible accounts
    public int applyInterestToEligible(double minThreshold, double interestRatePercent) {
        // Iterate through accounts; if balance >= minThreshold -> deposit interest amount
        // Return count of accounts that received interest
        return 0;
    }
}
```

3. Visual Execution Verification Snapshots

When your solution is completed and executed via Day6ChallengeDemo.java, your program output must match the expected visual execution trace below.

```
================================================
   HARAMAYA BANK: DAY 6 OOP CHALLENGE DEMO TRACE
================================================

--- 1. TESTING BANKACCOUNT VALIDATION ---
☑ [SUCCESS] Created Account ACC-101 (Abebe Bekele) with $500.00
☑ [SUCCESS] Created Account ACC-102 (Kebede Lemma) with $200.00
☑ [SUCCESS] Deposited $150.00 into ACC-101 -> New Balance: $650.00
☑ [SUCCESS] Withdrew $100.00 from ACC-101 -> New Balance: $550.00
❌ [REJECTED] Withdraw $700.00 from ACC-101 -> Overdraft Blocked! Balance ($550.00)

--- 2. TESTING ATOMIC ACCOUNT TRANSFER SERVICE ---
🔄 [TRANSFER] Initiating $200.00 transfer from ACC-101 to ACC-102...
☑ [SUCCESS] Withdrew $200.00 from ACC-101
☑ [SUCCESS] Deposited $200.00 into ACC-102
☑ [TRANSFER COMPLETE] Transfer of $200.00 from ACC-101 to ACC-102 Succeeded!

--- 3. TESTING BANK PORTFOLIO BATCH INTEREST ---
 Total Bank Liquidity  : $750.00
 Highest Balance Account: ACC-102 (Kebede Lemma) with $400.00
 Applying 5.0% Interest to Accounts with Balance >= $350.00...
 ☑ Interest $20.00 added to ACC-102 -> New Balance: $420.00
 Total Accounts Rewarded: 1
================================================
```

4. Student Architectural Reflection Checkpoints

Answer the following 4 conceptual questions to verify your understanding of object encapsulation.

Checkpoint 6.1 (Encapsulation Invariants): Why are fields like balance declared private instead of public? What security vulnerability occurs if an external class executes acc.balance = -99999.00; directly?

[Write your architectural analysis here]

Checkpoint 6.2 (Mutator Control vs Public Setters): Why does BankAccount omit a public setBalance(double b) setter method and instead require mutations to pass through deposit() and withdraw()?

[Write your architectural analysis here]

Checkpoint 6.3 (Memory Topography - Stack vs Heap): When BankAccount acc1 = new BankAccount("ACC-101", "Abebe", 500.0); executes, what specifically is stored on the Stack frame vs the dynamic Heap?

[Write your architectural analysis here]

Checkpoint 6.4 (Transaction Atomicity): In AccountTransferService, why must withdrawal occur BEFORE deposit? What corrupted state occurs if deposit is executed first but withdrawal fails due to insufficient funds?

[Write your architectural analysis here]

5. Evaluation Sheet & Grading Rubric

Evaluation Criteria	Performance Expectation	Max Marks	Score
Challenge 1: BankAccount Class	Encapsulation, Constructor validation, deposit & withdraw logic	**3.0 Marks**	
Challenge 2: Transfer Service	Null checks, self-transfer prevention, atomic execution	**2.5 Marks**	
Challenge 3: Portfolio Engine	Liquidity calculation, max finder, batch interest processing	**2.0 Marks**	
Reflection Checkpoints	Accurate technical analysis of memory, invariants, & atomicity	**1.5 Marks**	
Conduct & Code Quality	Compilation without warnings, standard naming, oral defense	**1.0 Mark**	

Signature: _____ Date: _____ Total Score: _____ / 10.0